NEXUS GESTÃO

Documento técnico de estruturação, arquitetura e execução

Guia para compreender a organização do repositório, iniciar as aplicações e demonstrar o protótipo.

Projeto acadêmico • Baseado nas informações públicas do repositório GitHub

Repositório: github.com/PedroHBassi/Projeto-Nexus

1. Visão geral do projeto

O Nexus Gestão é um protótipo de sistema de gestão com uma interface Web e uma API REST. A interface é construída com Spring Boot e Thymeleaf; a API também utiliza Spring Boot. O repositório descreve bancos H2 separados para Web e API nesta versão inicial.

Objetivo funcional

Organizar informações em módulos administrativos e oferecer uma interface para consultar e operar os dados de funcionários, contas do mês, veículos e serviços.

Escopo descrito no repositório

Módulo	Finalidade na interface
Funcionários	Área de cadastro/consulta de registros de funcionários.
Contas do mês	Área para organizar contas e informações mensais.
Veículos	Área de controle de informações de veículos.
Serviços	Área de organização dos serviços cadastrados.

Importante: este documento descreve a estrutura declarada no README público. Detalhes de classes, endpoints e regras de negócio devem ser confirmados nos arquivos-fonte atuais.

2. Estrutura do repositório

O repositório está organizado em dois projetos principais, além de documentação e materiais de apresentação.

```
Projeto-Nexus/
■■■ api/                          # Aplicação de API REST (porta 8081)
■■■ web/                          # Aplicação Web Thymeleaf (porta 8080)
■■■ README.md                     # Visão geral e instruções
■■■ referencia_visual_prototipo.png
■■■ Apresentação Nexus.pptx       # Nome atual pode variar no Git
```

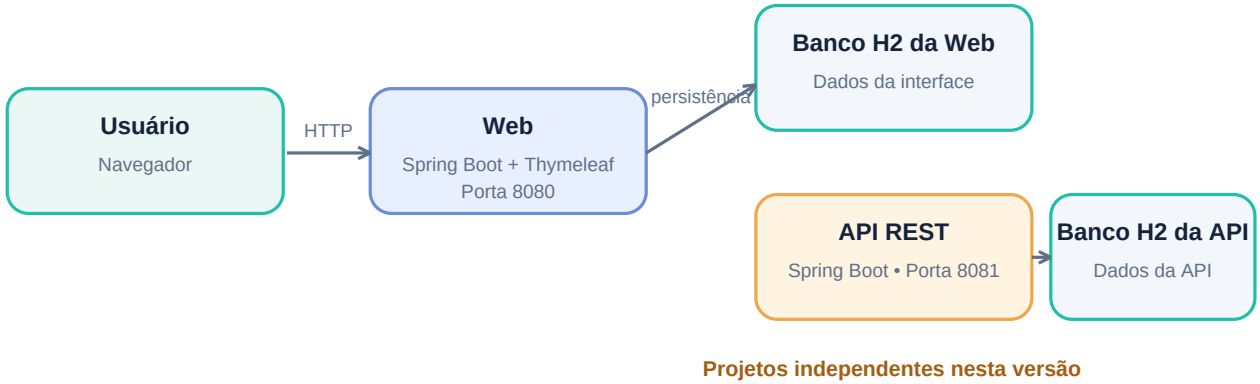
O README do GitHub pode conter nomes de apresentação antigos. O nome de um arquivo local não altera o funcionamento da aplicação; apenas é necessário que o Git registre a mudança de nome corretamente.

Papéis das partes

Parte	Responsabilidade
web	Aplicação acessada pelo navegador. Usa Spring Boot + Thymeleaf e tem configuração para a porta 8080.
api	Aplicação que expõe endpoints REST e está configurada para a porta 8081.
H2	Banco de dados em memória usado em desenvolvimento; a documentação informa instâncias separadas para Web e API.
README / materiais	Documentam o projeto e apoiam apresentação e execução.

3. Arquitetura em diagrama

O desenho abaixo representa a separação informada no repositório. Ele não afirma que a Web consome a API: na versão documentada, as aplicações são independentes e mantêm bancos H2 separados.



A Web e a API usam bancos separados; a integração HTTP entre elas não deve ser presumida.

Conceitos importantes

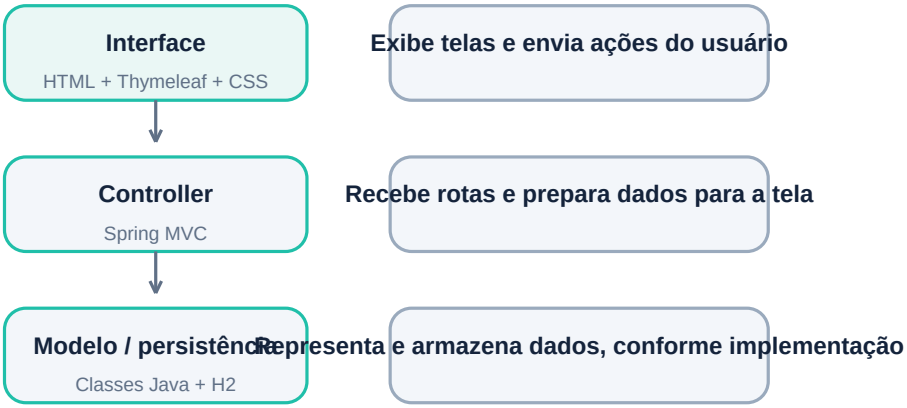
Spring Boot: facilita a inicialização e configuração das aplicações Java. **Thymeleaf:** permite gerar páginas HTML com dados preparados pelo servidor. **API REST:** disponibiliza recursos por rotas HTTP. **H2:** banco leve adequado a protótipos e desenvolvimento.

Limite atual a comunicar na apresentação

Como a Web e a API estão separadas e usam bases distintas, não apresente como fato que um cadastro feito na interface é enviado para a API. Para afirmar isso, é necessário verificar uma chamada HTTP no código e testar o fluxo.

4. Fluxo conceitual da aplicação Web

Em uma aplicação Spring MVC com Thymeleaf, o fluxo comum é: navegador solicita uma rota; o controller trata a requisição; dados são obtidos ou preparados; o Thymeleaf renderiza a página HTML e a resposta volta ao navegador.



Esquema conceitual: os nomes e camadas exatos devem ser confirmados nos arquivos-fonte.

Como explicar os arquivos Java (guia geral)

- **Classe principal:** inicia a aplicação Spring Boot.
- **Controller:** define rotas e decide qual página ou resposta retornar.
- **Modelo/entidade:** representa informações do domínio, quando usada.
- **Repository:** concentra operações de persistência, se o projeto usar essa camada.
- **Service:** reúne regras de negócio, quando essa camada existe.
- **application.properties:** guarda configurações como porta, banco e opções do Spring.

Essas são funções típicas do ecossistema Spring; confira a implementação do projeto antes de atribuir cada responsabilidade a um arquivo específico.

5. Pré-requisitos e preparação

Antes de executar, instale e confira os requisitos abaixo:

Requisito	Como conferir
Java 17 ou superior	No PowerShell: `java -version`
Maven	No PowerShell: `mvn -version`
Repositório local	Abra o terminal na pasta raiz que contém as pastas `api` e `web`.

Se o comando `mvn` não for reconhecido, verifique se o Maven está instalado e configurado no PATH. Se o projeto incluir o Maven Wrapper (`mvnw.cmd`), pode ser possível usá-lo no lugar de `mvn`.

6. Execução passo a passo

Abra dois terminais do PowerShell na pasta raiz do repositório. Mantenha cada processo rodando em seu próprio terminal.

Terminal 1 — iniciar a API

```
cd api
mvn spring-boot:run
```

A API deve iniciar na porta 8081, conforme o README. Aguarde a mensagem de inicialização sem erros.

Terminal 2 — iniciar a aplicação Web

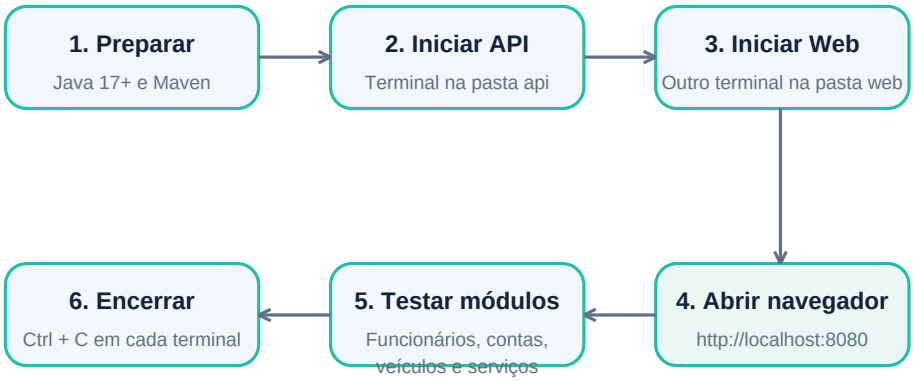
```
cd web
mvn spring-boot:run
```

A aplicação Web deve iniciar na porta 8080. Abra no navegador: http://localhost:8080.

Se você já estiver dentro da pasta `web`, não execute `cd web` novamente. Se os diretórios tiverem nomes diferentes na sua cópia local, adapte o comando ao nome real da pasta.

7. Fluxo de execução visual

Use este fluxo como checklist para iniciar e demonstrar o protótipo.



A API pode ser iniciada para testes próprios; a interface fica disponível na porta 8080.

Para encerrar as aplicações

Volte a cada terminal onde uma aplicação está rodando e pressione **Ctrl + C**. Fechar apenas a aba do navegador não encerra o servidor Spring Boot.

Verificação de funcionamento

1. A aplicação Web abre em `http://localhost:8080`.
2. O dashboard aparece sem erro de template.
3. É possível navegar pelos módulos exibidos.
4. As operações que o protótipo oferece respondem conforme esperado.
5. Se testar a API, use as rotas realmente definidas no código; não presuma endpoints específicos sem consultá-los.

8. Problemas comuns e diagnóstico

Sintoma	O que verificar
`mvn` não é reconhecido	Instalação e variável PATH do Maven; confira também se existe `mvnw.cmd`.
Porta 8080 ou 8081 ocupada	Encerre o processo que já usa a porta ou altere a configuração da aplicação, se apropriado.
Erro ao renderizar página Thymeleaf	Leia a mensagem completa e confira nomes de atributos, propriedades e variáveis referenciadas no HTML.
CSS não aparece	Confira o caminho do arquivo em `static`, os links no HTML e erros de carregamento no navegador.
API inicia, mas a Web não mostra seus dados	Nesta versão documentada, os projetos usam bancos separados e não se deve presumir integração automática.
`git push` não publica o PowerPoint	Confira `git status`, `git remote -v` e se o commit foi concluído. Nome com acento deve ser passado entre aspas no PowerShell.

9. Checklist para apresentação

Antes de apresentar ao professor ou à turma:

- ■ Confirmar que Java e Maven estão disponíveis.
- ■ Iniciar API e Web em terminais separados.
- ■ Abrir `http://localhost:8080`.
- ■ Mostrar o dashboard e os módulos disponíveis.
- ■ Explicar a diferença entre a interface Web e a API REST.
- ■ Dizer com clareza que, na versão documentada, os bancos H2 são separados.
- ■ Demonstrar apenas operações que foram testadas de verdade.
- ■ Preparar uma explicação breve das tecnologias usadas.

Resumo para falar em voz alta

“O Nexus Gestão é um protótipo de gestão desenvolvido com Java e Spring Boot. Ele está organizado em duas aplicações: uma interface Web com Thymeleaf, executada na porta 8080, e uma API REST na porta 8081. Nesta versão inicial, as aplicações são independentes e usam bancos H2 separados. Para executar, é necessário ter Java 17 ou superior e Maven, iniciar cada projeto em um terminal e acessar a interface pelo navegador.”

10. Referência e observação

Fonte principal: README público do repositório Projeto-Nexus no GitHub:
https://github.com/PedroHBassi/Projeto-Nexus

Este documento foi preparado com base na estrutura e nas instruções públicas do README. A listagem completa dos arquivos-fonte internos não foi verificada linha por linha durante a geração; por isso, os diagramas de camadas são conceituais e os comandos devem ser conferidos com a estrutura local do projeto.